

Cross-Traffic Intersection Management for Autonomous Vehicles

Week 3 - First Abstract Implementation

1 Scenario and Objective

The project realises an autonomous intersection in which a Road-Side Unit (RSU) coordinates the crossing of cooperative vehicles, replacing fixed traffic lights by an on-demand grant scheme. The geometry and the control contract are taken directly from the verified UPPAAL timed-automata model:

- **Topology.** Main street runs East-West; the side street runs North-South. The RSU owns the conflict zone.
- **Server capacity (queuing theory).** M/M/1 permits a single vehicle in the conflict zone at a time; M/M/2 permits two vehicles provided they are non-conflicting.
- **Priority order.** (1) emergency vehicle, (2) main-street vehicle, (3) side-street vehicle, (4) right turn before left turn inside a conflict.
- **Communication.** V2I: vehicle $\rightarrow$ RSU request (req/emg_req); I2V: RSU $\rightarrow$ vehicle grant plus an emergency yield broadcast (i2v_yield); V2V: vehicles broadcast intersection enter/leave.
- **Continuous arrival.** After leaving, every vehicle re-requests after its own arrival period, producing a cyclic, never-terminating workload.

Objective of Week 3. Following the course plan (Week 3: first abstract implementation), this milestone delivers a well-defined mapping from the UPPAAL model to executable C core logic. The implementation is intentionally single-threaded and deterministic; the FreeRTOS tasking and the hardware partitioning (ModelSim) are added in the later HW/SW-codesign milestones (Weeks 4 and 5). The abstract implementation must reproduce the same behaviour and the same safety guarantees that were model-checked in UPPAAL.

2 Recap of Weeks 1-2

Week 1 (requirements and use-cases) fixed the functional contract: the RSU must serialise conflicting movements, prioritise emergency and main-street traffic, support both M/M/1 and M/M/2 service, and rely solely on V2I/I2V/V2V messages. Week 2 (models and diagrams) captured this contract as a UPPAAL network of timed automata with two templates - Vehicle and RSU_Controller - instantiated as one RSU and six vehicles (Table 1). The model was verified against six queries (Section 5), all of which hold. Week 3 takes that verified model as its specification.

Table 1: Vehicle fleet instantiated in the UPPAAL system section and reproduced verbatim in the C implementation.

ID	Origin	Turn	Emerg.	Period	Role
V0	East	Straight	yes	30	emergency vehicle
V1	East	Straight	no	6	main street
V2	West	Straight	no	8	main street, opposite (M/M/2 partner of V1)
V3	North	Straight	no	10	side street
V4	North	Right	no	12	critical right-turn vehicle
V5	South	Left	no	12	critical left-turn vehicle

3 Mapping Methodology

The UPPAAL declaration block is already written in a C-like action language, so the mapping is structural rather than a re-design. Table 2 states the correspondence; every UPPAAL artefact has exactly one counterpart in the C source, which is the "well-defined mapping from models" required by the course brief.

4 Abstract Implementation Architecture

4.1 Vehicle finite-state machine

Each vehicle is the UPPAAL Vehicle automaton: it idles for its arrival period, sends a V2I request, waits for a grant, announces entry (V2V), occupies the conflict zone for crossTime, announces departure (V2V), and returns to idle. The local clock x[i] reproduces the UPPAAL clock; committed entry/leave steps are executed in the same tick.

4.2 RSU admission control

The RSU is stateless apart from the shared queue arrays. The admission predicate canGrant is the heart of the controller and is identical to the model: it blocks normal traffic while an emergency waits, enforces the server capacity, and yields to any higher-priority grantable candidate.

Listing 1: Admission predicate, mapped one-to-one from UPPAAL.

```
bool canGrant(int i) {
    if (!waiting[i]) return false;
    /* If an emergency is waiting, no normal vehicle gets a new grant. */
    if (emergencyWaiting() && emergency[i] == 0) return false;
    if (!capacityAvailableFor(i)) return false; /* M/M/1 or M/M/2 */
    if (existsBetterGrantable(i)) return false; /* strict priority */
    return true;
}
```

4.3 Server capacity: M/M/1 vs. M/M/2

The single switch MM_MODE selects the queuing model, exactly as the const int MM_MODE line in the UPPAAL file:

Listing 2: Capacity rule realising the M/M/1 and M/M/2 servers.

```
bool capacityAvailableFor(int i) {
    if (MM_MODE == MM1) return crossingCount == 0; /* one server */
    if (crossingCount >= 2) return false; /* two servers */
    return !conflictsWithCrossing(i); /* 2nd car only if non-conflicting */
}
```

Table 2: Well-defined mapping: UPPAAL model construct -> C core logic.

UPPAAL construct	C implementation
Global const/enums (EAST..SOUTH, RIGHT..LEFT, N, MM_MODE)	enum / #define; MM_MODE set from the command line.
Global arrays waiting, crossing, origin, turn, emergency, crossingCount, emergencyNotified	Identically named file-scope arrays / variables.
Helper functions isMainRoad, oppositeDirection, turnRank, priorityValue, betterPriority	Copied verbatim (same names, same bodies).
Conflict functions conflict, conflictsWithCrossing, noCollision	Copied verbatim.
Grant functions emergencyWaiting, capacityAvailableFor, existsBetterGrantable, canGrant	Copied verbatim.
Update functions requestVehicle, grantVehicle, finishVehicle	Copied verbatim.
Vehicle template (locations Idle, WaitingInQueue, V2V_Enter, Crossing, V2V_Leave)	loc_t enum + per-vehicle local clock x[i]; one switch per location in the scheduler loop.
RSU_Controller template (req/emg_req/grant/leave + yield)	Phases (a)-(d) of the simulation loop.
Clock x, invariants x<=period, x<=crossTime, guards x>=period, x>=crossTime	Discrete tick counter x[i]; one tick = one UPPAAL time unit; guards become integer comparisons.
Committed locations V2V_Enter / V2V_Leave	Processed in the same tick (zero elapsed time), preserving UPPAAL committed-location semantics.
Synchronisations req!/? , grant!/? , v2v_enter!, v2v_leave!, i2v_yield!	Direct function calls plus a console event log line (the message is the log entry).
Non-deterministic select i:int[0,5] with guard canGrant(i)	Deterministic priority scan: grant the best canGrant candidate, repeat until none (Section 4).
Queries A[]/E<>	Run-time assertions + reachability statistics (Section 5).

4.4 Deterministic resolution of the UPPAAL select

UPPAAL grants a non-deterministically chosen vehicle for which canGrant holds. To obtain a reproducible simulation, the C scheduler repeatedly grants the best-priority grantable vehicle until none remains. Because canGrant already rejects any candidate that has a better grantable competitor, the loop visits exactly the same grant set as the model, only in a fixed order.

Listing 3: Priority-ordered grant phase (one RSU control cycle).

```
do {
    granted = false; int best = -1;
    for (int i = 0; i < N; i++)
        if (canGrant(i) && (best < 0 || betterPriority(i, best))) best = i;
    if (best >= 0) {
        grantVehicle(best);                /* I2V grant + state update */
        loc[best] = LOC_CROSS; x[best] = 0; /* committed V2V_Enter, zero time */
        granted = true;
    }
} while (granted);
```

5 Verification Correspondence

The six UPPAAL queries become run-time checks. The three safety invariants are asserted after every tick; the three reachability properties are confirmed by observing the event log and the run-summary counters. Table 3 lists the result; all checks held over the simulated horizon (and the safety assertions never fired).

Table 3: UPPAAL queries and their run-time counterpart in the abstract implementation.

UPPAAL query	Run-time counterpart	Holds
--------------	----------------------	-------

A[] not deadlock	cyclic loop always has an enabled action; the simulation never stalls	yes
A[] noCollision()	assert(noCollision()) every tick	yes
A[] crossingCount <= MM_MODE	assert(crossingCount <= MM_MODE) every tick	yes
A[] not (crossing[4] && crossing[5])	assert(!(crossing[4] && crossing[5])) every tick	yes
E<> crossing[0]	emergency V0 is granted (log + counter emergency_grants)	yes
E<> crossing[1] && crossing[2]	V1 and V2 cross together in M/M/2 (cross=2 in the log)	yes

6 Results

The program is built and executed in both modes:

```
gcc -std=c11 -Wall -O2 intersection_rsu.c -o intersection_rsu
./intersection_rsu mm1      # M/M/1
./intersection_rsu mm2      # M/M/2
```

6.1 M/M/2: concurrency and emergency pre-emption

The M/M/2 trace below shows the two key behaviours. At $t = 8$, vehicle V2 is granted while V1 is still crossing, so two non-conflicting main-street vehicles share the conflict zone (cross=2) - the witness for query $E \langle \rangle \text{crossing}[1] \ \&\& \text{crossing}[2]$. At $t = 30$ the emergency vehicle V0 requests; the RSU immediately broadcasts I2V_YIELD, withholds all normal grants, lets the in-flight crossing (V3) finish, and at $t = 33$ grants V0 exclusively (cross=1).

```
[t= 6] V1 REQ      (EAST STRAIGHT) -> RSU
[t= 6] RSU GRANT   -> V1 (EAST STRAIGHT) cross=1
[t= 8] V2 REQ      (WEST STRAIGHT) -> RSU
[t= 8] RSU GRANT   -> V2 (WEST STRAIGHT) cross=2  <-- M/M/2 concurrency
[t= 11] V1 DONE    -> Idle cross=1
[t= 12] V4 REQ      (NORTH RIGHT) -> RSU
[t= 12] V5 REQ      (SOUTH LEFT) -> RSU
[t= 14] RSU GRANT   -> V4 (NORTH RIGHT) cross=1  <-- V4 only, not V5
[t= 27] RSU GRANT   -> V3 (NORTH STRAIGHT) cross=1
[t= 30] V0 EMG_REQ  (EAST STRAIGHT) -> RSU
[t= 30] RSU I2V_YIELD broadcast (emergency approaching) <-- I2V yield
[t= 32] V3 DONE    -> Idle cross=0
[t= 33] RSU GRANT   -> V0 (EAST STRAIGHT EMERGENCY) cross=1 <-- exclusive
```

6.2 M/M/1: strict serialisation

In M/M/1 the same workload is serialised: V2's request at $t = 8$ is queued and only granted at $t = 12$ after V1 leaves; crossingCount never exceeds 1.

```
[t= 6] RSU GRANT   -> V1 (EAST STRAIGHT) cross=1
[t= 8] V2 REQ      (WEST STRAIGHT) -> RSU          <-- queued, not granted
[t= 11] V1 DONE    -> Idle cross=0
[t= 12] RSU GRANT   -> V2 (WEST STRAIGHT) cross=1
```

6.3 Quantitative comparison

Over an identical 80-tick horizon the two modes give:

Table 4: Throughput of the abstract implementation (identical 80-tick horizon).

Metric	M/M/1	M/M/2
Total requests served	16	21
Total grants	13	18
Emergency grants	2	2
Max concurrent crossings	1	2
Ticks with two crossings	0	23
Safety assertions fired	0	0

M/M/2 increases throughput (18 vs. 13 grants) purely by overlapping compatible movements, while every safety invariant continues to hold - the expected benefit of the second server.

6.4 Observed limitation: starvation of the lowest-priority movement

The strict static-priority discipline reveals a fairness issue that the reachability queries do not forbid: vehicle V5 (side-street left turn, the lowest non-emergency priority) is never granted, in either mode, even when the horizon is extended to 400 ticks. Under continuous arrivals the higher-priority movements (V1/V2 main street, V3 side-straight, V4 side-right) always pre-empt it. This is consistent with the model - the UPPAAL queries assert only safety and the possibility of crossing, not liveness for every vehicle - and motivates the next milestone.

7 Conclusion and Outlook

Week 3 delivers a compilable, deterministic C core that is a one-to-one mapping of the verified UPPAAL model. It reproduces M/M/1 and M/M/2 service, the full priority order, emergency pre-emption with I2V yield, the critical North-right / South-left mutual exclusion, and the V2I/I2V/V2V message flow, while continuously re-checking the model's safety invariants at run time.

For Week 4 (HW/SW codesign proposal) the natural partition is to keep this admission logic (conflict, canGrant, queue management) in software and to offload the cycle-critical, parallel noCollision / conflict-matrix evaluation to hardware (ModelSim/VHDL), exposing it to the RSU as a single combinational "safe-to-grant" check. The starvation observation above will additionally be addressed by adding a fairness/aging term to the priority so that low-priority side-street movements are guaranteed eventual service - to be carried into the FreeRTOS implementation in Week 5.

The complete, commented source file `intersection_rsu.c` accompanies this report and builds with `gcc -std=c11 -Wall -O2`.